APCS 程式設計系列 · U08 · 實作中高級 · C++版

排序與二分搜尋 重點講義

泡沫 / 選擇 / 插入排序 · 內建排序與自訂排序 · 穩定性 · $O(n^2)$ 與 $O(n \log n)$ · 二分搜尋 · 上下界 · 二分答案

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）

看這個單元的講解影片（YouTube）

【APCS C++】U08 排序與二分搜尋｜排序、二分搜尋、二分答案｜8 分鐘重點講解

手機掃描 QR code，或點這裡直接看 https://youtu.be/jjGFZ6MwCnw

1. 三種基本排序：識讀題最愛考「某一輪之後」

C++ **泡沫排序：每一輪把最大的推到最後**

```cpp
int a[5] = {5, 1, 4, 2, 8};
int n = 5, swaps = 0;
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1 - i; j++)   // 後面 i 個已經排好
        if (a[j] > a[j + 1]) {
            swap(a[j], a[j + 1]);
            swaps++;
        }
    for (int k = 0; k < n; k++) cout << a[k] << " ";
    cout << "\n";
}
cout << "swaps = " << swaps << "\n";
```

輸入	輸出
（不需要輸入）	1 4 2 5 8 1 2 4 5 8 1 2 4 5 8 1 2 4 5 8 swaps = 4

C++ 選擇排序：每一輪挑最小的換到前面

```cpp
int a[5] = {5, 1, 4, 2, 8}, n = 5;
for (int i = 0; i < n - 1; i++) {
    int m = i;                      // 最小值的位置
    for (int j = i + 1; j < n; j++)
        if (a[j] < a[m]) m = j;
    swap(a[i], a[m]);               // 每一輪最多交換一次
    for (int k = 0; k < n; k++) cout << a[k] << " ";
    cout << "\n";
}
```

輸入	輸出
（不需要輸入）	1 5 4 2 8 1 2 4 5 8 1 2 4 5 8 1 2 4 5 8

C++ 插入排序：把新的數插進前面排好的部分

```cpp
int a[5] = {5, 1, 4, 2, 8}, n = 5;
for (int i = 1; i < n; i++) {
    int key = a[i], j = i - 1;
    while (j >= 0 && a[j] > key) {   // 比 key 大的往後挪
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = key;
    for (int k = 0; k < n; k++) cout << a[k] << " ";
    cout << "\n";
}
```

輸入	輸出
（不需要輸入）	1 5 4 2 8 1 4 5 2 8 1 2 4 5 8 1 2 4 5 8

排序法	每一輪做什麼	識讀時的觀察重點
泡沫排序	相鄰兩個比，前面大就交換；第 i 輪結束，第 i 大的數歸位到最後	第 k 輪後，「最後 k 個」一定是最大的 k 個且排好
選擇排序	在還沒排的部分找最小值，跟最前面交換	第 k 輪後，「前 k 個」一定是最小的 k 個；每輪最多交換 1 次
插入排序	把第 i 個數往前插，插進前面已排好的部分	第 k 輪後，「前 k + 1 個」排好了，但不一定是全體最小的

交換次數 = 逆序對數

- **逆序對**：位置在前、數值卻比較大的一對 (i < j 且 a[i] > a[j])。
- 泡沫排序每交換一次，剛好消掉一個逆序對，所以**總交換次數 = 逆序對數**；插入排序「往後挪」的次數也一樣。
- 例：5 1 4 2 8 的逆序對有 (5,1)(5,4)(5,2)(4,2) 共 4 對，泡沫排序就交換 4 次。

陷阱提醒

【錯】搞錯是「第幾輪」：題目的第 1 輪，常常是程式裡 i = 0 那一圈。
【錯】選擇排序找的是「位置」m，最後交換 a[i] 和 a[m]；只記數值不記位置就換不回去。

2. 內建排序：考場上一律用它

C++ 內建排序：小到大、大到小

```cpp
vector<int> v = {5, 1, 4, 2, 8};
sort(v.begin(), v.end());                    // 小到大
for (int x : v) cout << x << " ";
cout << "\n";
sort(v.begin(), v.end(), greater<int>());    // 大到小
for (int x : v) cout << x << " ";
cout << "\n";
```

輸入	輸出
（不需要輸入）	1 2 4 5 8 8 5 4 2 1

陷阱提醒

【錯】一般陣列寫 sort(a, a + n)：範圍是「開頭到結尾的下一格」，寫成 a + n - 1 會漏掉最後一個。
【錯】只排前 k 個？sort(a, a + k)。排第 1～n 格（1-based）？sort(a + 1, a + n + 1)。

3. 自訂排序：多個鍵、比較函式與穩定性

C++ 多鍵排序：分數高的在前，同分座號小的在前

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Stu { int id, score; };

bool cmp(const Stu &a, const Stu &b) {
    if (a.score != b.score) return a.score > b.score;  // 分數高在前
    return a.id < b.id;                                // 座號小在前
}

int main() {
    int n;
    cin >> n;
    vector<Stu> s(n);
    for (int i = 0; i < n; i++) {
        s[i].id = i + 1;
        cin >> s[i].score;
    }
    sort(s.begin(), s.end(), cmp);
    for (auto &t : s) cout << t.id << " ";
    cout << "\n";
}
```

輸入	輸出
5 80 95 80 70 95	2 5 1 3 4

比較規則怎麼寫

- cmp(a,b)回傳true代表「a要排在b前面」。多個鍵：第一個鍵不同就比第一個，相同才比第二個。
- 也可以用 lambda 直接寫在 sort 裡：sort(v.begin(), v.end(), [](int x, int y) { return x > y; });
- C 題本用 **qsort**：比較函式回傳「負數」代表 a 在前、0 代表相同、正數代表 b 在前。

C++ C 題本會看到的 qsort

```cpp
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *p, const void *q) {
    int a = *(const int *)p, b = *(const int *)q;
    return a - b;        // 負數：a 在前；改成 b - a 就是大到小
}

int main() {
    int a[5] = {5, 1, 4, 2, 8};
    qsort(a, 5, sizeof(int), cmp);
    for (int i = 0; i < 5; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

輸入	輸出
（不需要輸入）	1 2 4 5 8

數字很大（接近 int 上限）時 a - b 可能溢位，安全寫法是 return (a > b) - (a < b);

穩定排序：鍵相同時，保不保留原本的先後？

- C++ 的 sort **不保證穩定**：分數相同的人，排完先後可能被打亂。需要時改用 **stable_sort**，或把座號也寫進比較規則。
- C 的 qsort 也不保證穩定。泡沫排序、插入排序是穩定的；選擇排序不穩定。

陷阱提醒

【錯】cmp 寫成 a.score >= b.score：相等時也回傳 true，違反規則，sort 可能跑錯甚至當掉。一定用 > 或 <。

4. $O(n^2)$ 和 $O(n \log n)$：資料一多就差很多

做法	時間複雜度	$n = 2\times10^5$ 大約	結論
泡沫 / 選擇 / 插入排序	$O(n^2)$	2×10^{10} 次	一定超時
內建 sort / sorted	$O(n \log n)$	約 3.6×10^6 次	輕鬆
q 次詢問，每次從頭找	$O(nq)$	4×10^{10} 次	一定超時
先排序，每次二分搜尋	$O((n + q) \log n)$	約 7×10^6 次	輕鬆

- $\log(2\times10^5)$（以 2 為底）大約 18：二分搜尋找 20 萬筆資料，最多看 18 次左右。
- 基本排序不是沒用：識讀題會考它的過程；插入排序在資料「幾乎排好」時很快。
- 實作題看到 $n \leqq 10^5$ 以上又要排序，直接用內建排序。

5. 二分搜尋：在排好序的資料裡，每次砍掉一半

C++ 二分搜尋：在排好序的陣列找 x

```cpp
int a[10] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
int x = 23, l = 0, r = 9, ans = -1;
while (l <= r) {
    int mid = l + (r - l) / 2;       // 不會溢位的寫法
    if (a[mid] == x) { ans = mid; break; }
    else if (a[mid] < x) l = mid + 1;   // 答案在右半
    else r = mid - 1;                   // 答案在左半
}
cout << ans << "\n";
```

輸入	輸出
（不需要輸入）	5

陷阱提醒 二分搜尋三大陷阱

【錯】無窮迴圈：寫 l = mid（沒有 + 1），當 l、r 相鄰時 mid 算出來還是 l，區間永遠不會縮小。
【錯】mid 溢位：l、r 接近 2×10^9 時，l + r 超過 int。改寫成 l + (r - l) / 2。
【錯】邊界：while (l <= r) 配 r = n - 1、r = mid - 1；while (l < r) 配 r = n、r = mid。兩套不要混著用。
【對】卡住時，拿 l、r 相鄰（例如 l = 4、r = 5）的情況代進去跑一次，看區間有沒有變小。

6. 上下界：第一個 >= x、第一個 > x

C++ 內建上下界：x 出現幾次

```cpp
vector<int> a = {1, 3, 3, 3, 5, 7, 7, 9};
// lower_bound：第一個 >= 3；upper_bound：第一個 > 3
int lo = lower_bound(a.begin(), a.end(), 3) - a.begin();
int hi = upper_bound(a.begin(), a.end(), 3) - a.begin();
cout << lo << " " << hi << " " << hi - lo << "\n";
```

輸入	輸出
（不需要輸入）	1 4 3

想找的	C++	Python
第一個 >= x 的位置	lower_bound(..., x)	bisect_left(a, x)
第一個 > x 的位置	upper_bound(..., x)	bisect_right(a, x)
x 出現幾次	upper - lower	bisect_right - bisect_left
落在 [L, R] 的個數	upper(R) - lower(L)	bisect_right(a, R) - bisect_left(a, L)

C++ 手寫 lower_bound：第一個 >= x 的位置

```cpp
int a[8] = {1, 3, 3, 3, 5, 7, 7, 9};
int n = 8, x = 4;
int l = 0, r = n;              // 答案可能是 n（全部都 < x）
while (l < r) {
    int mid = l + (r - l) / 2;
    if (a[mid] >= x) r = mid;  // mid 可能就是答案，留著
    else l = mid + 1;
}
cout << l << "\n";
```

輸入	輸出
（不需要輸入）	4

- 找不到 x 時，lower_bound 剛好給出「x 應該插入的位置」，這比單純回答「有沒有」更有用。
- C 題本沒有 lower_bound，識讀題會用手寫的 while 迴圈考你最後 l 停在哪裡。

7. 二分答案：答案有單調性，就二分「答案」

- 題目問「**最大的**…使得可以」或「**最小的**…使得可以」，而且答案越小越容易成立（或越大越容易）：答案有**單調性**。
- 步驟：(1) 寫 ok(x)：答案是 x 行不行？通常貪心掃一遍，O(n)。(2) 定出答案範圍 l、r。(3) 二分找出 ok 由「行」變「不行」的邊界。
- 總時間 O(n log(範圍))：範圍到 10^9 也只要二分約 30 次。

C++ 二分答案：切木頭

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
long long k, a[200005];

bool ok(long long len) {        // 每段長 len，切得出 k 段嗎？
    long long cnt = 0;
    for (int i = 0; i < n; i++) cnt += a[i] / len;
    return cnt >= k;
}

int main() {
    cin >> n >> k;
    for (int i = 0; i < n; i++) cin >> a[i];
    long long l = 1, r = *max_element(a, a + n), ans = 0;
    while (l <= r) {
        long long mid = l + (r - l) / 2;
        if (ok(mid)) { ans = mid; l = mid + 1; }  // 行：試更長
        else r = mid - 1;                          // 不行：縮短
    }
    cout << ans << "\n";
}
```

輸入	輸出
4 11 802 743 457 539	200

n 根木頭，長度分別是 802、743、457、539，要切出至少 11 段一樣長的木頭（剩下的丟掉），每段最長多長？長度 200 切得出 4 + 3 + 2 + 2 = 11 段，201 只有 10 段。

要找的	while (l < r) 的寫法	為什麼
最小的成立值（ok 前面不行、後面行）	mid = (l + r) / 2；ok 就 r = mid，否則 l = mid + 1	mid 向下取整，r = mid 一定會變小
最大的成立值（ok 前面行、後面不行）	mid = (l + r + 1) / 2；ok 就 l = mid，否則 r = mid - 1	要向上取整，不然 l = mid 會原地不動

陷阱提醒

【錯】寫 l = mid 卻用向下取整的 mid：l = 4、r = 5 時 mid = 4，l 永遠是 4，無窮迴圈。

【錯】答案範圍定錯：例如切木頭的 l 從 0 開始，ok(0) 會除以 0。

【錯】ok() 裡的計數可能超過 int（10^5 根長 10^9 的木頭切成長 1 的段），要用 long long。

讀完這份，你應該要會

- □ 給一個陣列，寫得出泡沫、選擇、插入排序「第 k 輪之後」的樣子
- □ 知道泡沫排序的交換次數 = 逆序對數
- □ 會用 sort 搭配 cmp 或 lambda 做多鍵排序，知道 cmp 不能寫 >=
- □ 知道 sort 不穩定、stable_sort 穩定
- □ 二分搜尋寫得出 while (l <= r) 版本，也知道 l = mid 為什麼會無窮迴圈
- □ 會用 lower_bound / upper_bound 算出 x 的個數與區間個數
- □ 看到「最大的…使得可以」，想得到二分答案 + 寫 ok(x)

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 @chuanglaomath

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）